

# FitTrack AI - Personalized fitness tracking and AI-powered fitness assistance.

|             |                                      |
|-------------|--------------------------------------|
| TEAM LEADER | HARSHINI E                           |
| EMAIL ID    | sac2427bca5777@ssacollegachennai.com |
| TEAM MEMBER | HARINI M                             |
| EMAIL ID    | sac2427bac5213@ssacollegechennai.com |
| TEAM MEMBER | JASRA J                              |
| EMAIL ID    | sac2427bac5454@ssacollegechennai.com |
| TEAM MEMBER | JENCY JAYA G                         |
| EMAIL ID    | sac2427bca5345@ssacollegechennai.com |

## DESCRIPTION

**FitTrack AI** is a secure, scalable, AI-powered RESTful backend for workout tracking and intelligent fitness guidance. Built with **Node.js**, **Express.js**, **MongoDB**, and **Mongoose**, it exposes a clean set of REST APIs supporting user authentication, workout management, workout search, and AI-driven fitness recommendations for a React frontend and other API clients.

### Architecture

The backend follows a modular **MVC architecture**, cleanly separating:

- **Routes** – API endpoint definitions
- **Controllers** – Request handling and business logic
- **Models** – Mongoose schemas for MongoDB collections
- **Middleware** – Authentication, validation, and error handling
- **Config** – Environment-based application settings
- **AI Services** – Dedicated Gemini integration layer

This separation of concerns keeps the codebase maintainable, testable, and easy to extend.

### Authentication & Security

- **JWT-based authentication** secures protected routes, ensuring only authenticated users can access workout and AI features.

- **Passwords are hashed with bcrypt.js** before being persisted to MongoDB, protecting user credentials even in the event of a data breach.
- **Authentication middleware** validates tokens on every protected request, decoupling security logic from route handlers.

### AI-Powered Features (Google Gemini Integration)

- **Personalized Workout Recommendations** – Users provide age, fitness goal, and experience level to receive a tailored workout plan.
- **AI Fitness Assistant Chatbot** – Answers natural-language fitness questions with concise, beginner-friendly responses.
- **Automated Fitness Insights** – Generates summaries from workout statistics (total workouts, average duration, calories burned) to help users understand their progress at a glance.

All AI functionality is isolated in a dedicated Gemini service module, keeping third-party API logic separate from core application code.

### Reliability & Maintainability

- **Input validation** on all endpoints to catch malformed requests early
- **Centralized error handling** for consistent, predictable API responses
- **Environment-based configuration** for seamless deployment across dev/staging/production
- **Protected routes** enforced uniformly via middleware

### Frontend Integration

The backend serves a React frontend, powering an end-to-end experience across authentication, dashboard analytics, workout management, AI recommendations, and chatbot interaction.

## Scenario-Based Case Study

Here's the background rewritten with Harshini as the user:

---

### Background

**Harshini** is a fitness enthusiast who trains regularly but lacks a simple, reliable way to manage her workout history. She needs a system that lets her:

- **Log workouts** as she completes them
- **Review past sessions** to track what she's done over time
- **Edit or delete incorrect entries** without hassle

- **View basic progress statistics** — such as workout frequency, duration, and calories burned — to understand how she's improving

Beyond tracking, Harshini also wants **personalized workout guidance** based on her goals and experience level, along with a **quick, conversational way to ask fitness-related questions** — all within the same application, without needing to search elsewhere or switch tools.

---

This background naturally maps to FitTrack AI's core capabilities:

| Harshini's Need        | FitTrack AI Feature                    |
|------------------------|----------------------------------------|
|                        |                                        |
| Log workouts           | Workout creation API                   |
| Review sessions        | Workout history / search               |
| Edit or remove records | Update / delete workout endpoints      |
| Progress statistics    | AI-generated fitness insights          |
| Personalized guidance  | Gemini-powered workout recommendations |
| Quick fitness Q&A      | AI Fitness Assistant chatbot           |

---

## Problem

Harshini currently faces several challenges in managing her fitness journey:

- **No centralized record-keeping** — workout data is scattered, making it difficult to maintain a single, reliable history.
- **Inaccurate tracking** — manual logging is error-prone, leading to inconsistent or incorrect workout records.
- **Lack of secure access control** — there is no authentication system to protect personal fitness data from unauthorized access.
- **Inefficient workout retrieval** — finding specific past workouts is slow and cumbersome without a proper search mechanism.
- **No personalization** — workout plans are generic, with no recommendations tailored to individual goals or experience level.
- **Limited visibility into progress** — there is no way to gain meaningful insights into overall performance trends over time.
- **Time-consuming manual calculations** — calculating calories burned and workout duration by hand is tedious and inefficient.

## Solution

FitTrack AI provides a centralized backend system that lets users securely register, authenticate, and manage their complete workout history in one place. Through RESTful APIs, users can **add, update, delete, retrieve, and search** workouts efficiently — eliminating scattered records and manual tracking errors.

By integrating **Google Gemini AI**, the application analyzes user profile data and workout statistics to deliver:

- **Personalized workout recommendations** based on age, fitness goal, and experience level
- **Automated fitness insights** derived from workout statistics — removing the need for manual calculation of calories burned or session duration
- **Conversational fitness guidance** through an AI-powered chatbot for quick, on-the-fly questions

To ensure user data remains secure and reliable, the backend implements:

| Safeguard                   | Purpose                                                              |
|-----------------------------|----------------------------------------------------------------------|
| JWT authentication          | Restricts access to verified, logged-in users                        |
| bcrypt password hashing     | Protects stored credentials from exposure                            |
| Protected middleware        | Enforces authentication on sensitive routes                          |
| MongoDB/Mongoose validation | Maintains data integrity and consistency                             |
| Environment variables       | Keeps sensitive configuration (API keys, secrets) out of source code |
| Centralized error handling  | Delivers consistent, predictable API responses                       |

# Usage

## User Usage

A registered user can create an account using the registration endpoint and securely log in using email and password. After successful authentication, the backend generates a JWT token that authorizes access to protected APIs.

After login, users can use the React application and authenticated APIs to:

- Create new workout records.
- View all previously completed workouts.
- Retrieve a workout by its unique identifier.
- Update existing workout information.
- Delete unwanted workout records.

- Search workouts using workout name, category, or workout date.
- Generate AI-powered workout recommendations.
- Obtain AI-generated fitness insights based on workout statistics.

## AI Module

The AI module communicates with Google Gemini through a **dedicated service layer**, keeping prompt preparation and AI communication logic fully separated from the controllers and the React frontend. This separation ensures the AI integration remains maintainable and easy to update independently of core application logic.

### 1. Personalized Workout Recommendations

Users provide basic profile information:

- **Age**
- **Fitness Goal**
- **Experience Level**

Based on this input, Gemini generates:

- **A personalized workout plan**
- **Weekly exercise suggestions**
- Recommendations tailored to **Beginner / Intermediate / Advanced** skill levels
- **Motivational guidance** to support consistency
- **Safety tips** relevant to the user's experience level

### 2. AI-Generated Fitness Insights

Users can submit their workout statistics:

- **Total Workouts**
- **Average Workout Duration**
- **Total Calories Burned**

The AI module analyzes these values and generates **personalized fitness insights**, helping users understand trends and progress without manual calculation.

### 3. AI Fitness Assistant Chatbot

Authenticated users can interact with an **AI Fitness Assistant chatbot**, submitting natural-language fitness questions and receiving **concise, AI-generated responses** — enabling quick guidance without leaving the application.

# Outcome

The completed AI FitTrack application brings authentication, workout management, progress tracking, and AI assistance into one fitness platform. Users can securely maintain their workout history, correct or remove records, view dashboard statistics, generate personalized recommendations, and communicate with the AI Fitness Assistant. JWT protection ensures that workout data is associated with the authenticated user.

The modular architecture supports future extensions such as nutrition tracking, BMI calculation, wearable-device integration, workout reminders, progress charts, and more detailed fitness analytics without major changes to the existing core modules.

## 1. Software Requirements

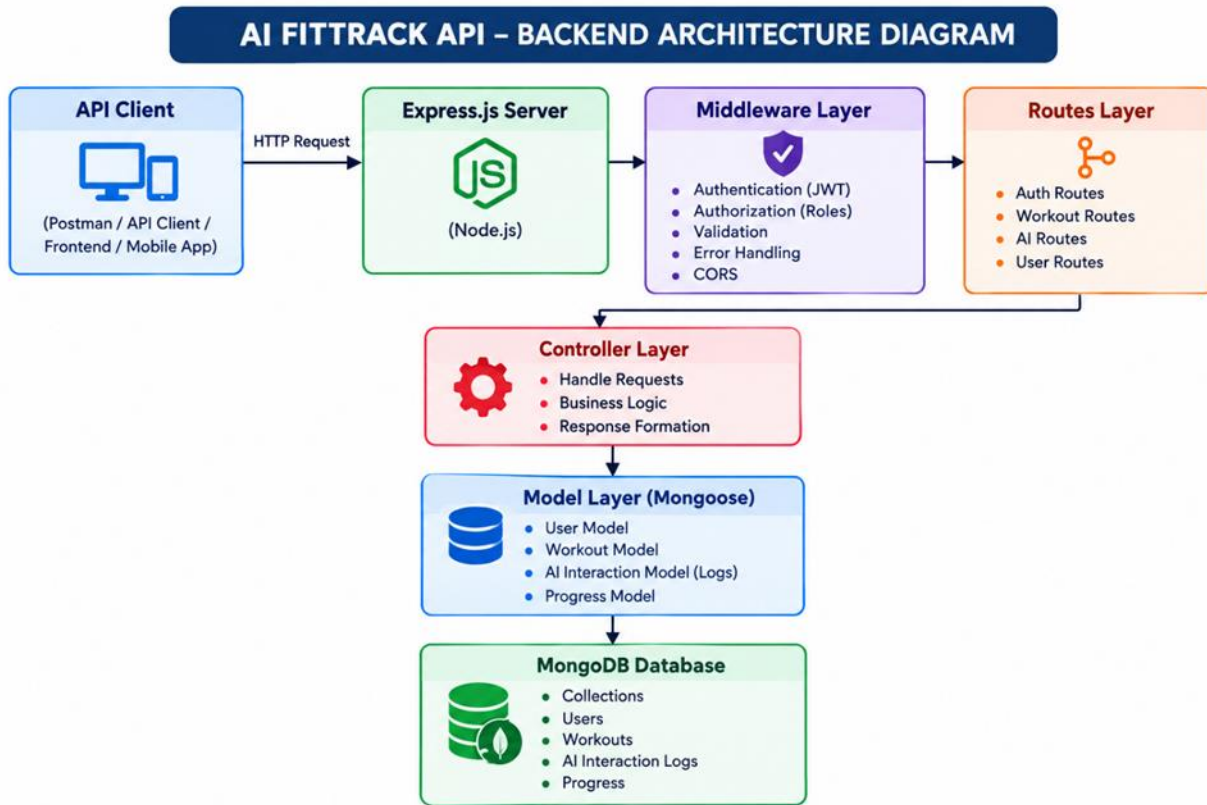
The FitTrack AI requires the following software components for development and execution. **Operating System:** Windows 10/11, macOS, or Linux (supports cross-platform operations).

- **Node.js (v16 or above):** Runtime ecosystem running server logic and routing infrastructure.
- **npm (v8 or above):** Node package manager to control operational dependencies.
- **Express.js:** Lightweight routing web framework to construct backend RESTful entry points.
- **users and workout records**
- **Postman:** API verification toolkit to assert schema validation responses across administrative route shields.
- **Code Editor:** Visual Studio Code or similar IDE environment.

## 2. Hardware Requirements

- **Processor:** Intel Core i5 (8th Gen or above) / AMD Ryzen 5 or equivalent.
- **RAM:** 8 GB minimum (16 GB recommended for concurrent instances of MongoDB, processing layers, and execution nodes).
- **Storage:** 1 GB of available disk workspace.

# Epic 1: Project Architecture



The FitTrack AI follows a layered architecture based on the MVC (Model–View–Controller) design pattern. The application separates routing, business logic, database interaction, middleware, and AI services into independent modules, making the project easier to maintain and scale.

When a client sends an HTTP request, the Express server receives it and forwards it to the corresponding route. Protected routes first pass through the authentication middleware, which validates the JWT token before granting access. Controllers process validated requests and invoke business logic or AI services as required. Database interactions are handled through Mongoose models, while Google Gemini AI is accessed through a dedicated service layer. Finally, the controller returns a structured JSON response to the client.

This layered approach improves modularity, security, and maintainability while ensuring that each component has a single responsibility.

## Overall Architecture Components

- Client Application (React / Mobile App / Postman)
- Express Server
- API Routes
- Authentication Middleware
- Controllers

- Services
- Google Gemini AI Service
- MongoDB Database
- Mongoose Models

Each component communicates through well-defined interfaces, ensuring loose coupling and easier future enhancements.

## Components Definition

Here's a polished version of the Architecture section — restructured for readability while preserving all technical detail:

---

### System Architecture

FitTrack AI follows a **modular, layered backend architecture** where each component has a single, well-defined responsibility. This separation of concerns ensures maintainability, scalability, and reliable request processing. Every incoming request passes through multiple layers — client, gateway, middleware, routing, controllers, services, and database — before a response is returned.

#### 1. Client Layer (Frontend / API Consumer)

The **React frontend** serves as the primary client, providing the user interface for registration, login, dashboard statistics, workout management, AI recommendations, and chatbot interaction. It communicates with the Express backend via HTTP requests and consumes JSON responses.

#### Responsibilities

- Sends API requests to the backend
- Displays workout data and statistics
- Stores the JWT token after successful login
- Invokes AI recommendation and chatbot APIs
- Displays success and error feedback to the user

#### APIs Consumed

- Authentication APIs
  - Workout APIs
  - AI Recommendation APIs
  - AI Fitness Insight APIs
-

## 2. Express Server Gateway

The **Express server** acts as the central gateway, handling incoming requests, configuring middleware, parsing request bodies, enabling CORS, and routing requests to the appropriate controllers.

### Responsibilities

- Starts and manages the HTTP server
- Parses incoming JSON request bodies
- Enables Cross-Origin Resource Sharing (CORS)
- Registers all application routes
- Connects global middleware
- Handles centralized error responses

### Request Flow

Receive HTTP Request → Validate Request Format → Forward to Routes → Return JSON Response

---

## 3. Authentication Middleware

Authentication middleware protects all secured endpoints by validating the JSON Web Token (JWT) on every protected request. Requests must include a valid `Authorization` header to proceed.

### Responsibilities

- Reads the `Authorization` header
- Extracts the JWT token
- Verifies the token signature
- Decodes user identity from the token
- Attaches authenticated user data to the request object
- Rejects unauthorized or invalid requests

### Benefits

- Prevents unauthorized access to sensitive endpoints
  - Secures user-specific workout data
  - Ensures only authenticated users reach protected APIs
  - Centralizes authentication logic, removing duplication from controllers
-

## 4. Route Layer

Routes define the application's API endpoints, mapping HTTP requests to their corresponding controller methods.

| Route Group    | Endpoints                                                                                              |
|----------------|--------------------------------------------------------------------------------------------------------|
| Authentication | Register User · Login User · Get User Profile                                                          |
| Workouts       | Add Workout · Get All Workouts · Search Workouts · Get Workout by ID · Update Workout · Delete Workout |
| AI             | Workout Recommendation · Fitness Insights                                                              |

## 5. Controller Layer

Controllers hold the application's core business logic — receiving requests, validating input, invoking services or database operations, and returning standardized JSON responses.

| Controller                | Responsibilities                                                                              |
|---------------------------|-----------------------------------------------------------------------------------------------|
| Authentication Controller | User registration, login, profile retrieval                                                   |
| Workout Controller        | Create, retrieve, update, delete, and search workout records                                  |
| AI Controller             | Sends prompts to Google Gemini, receives AI-generated output, returns responses to the client |

## 6. Service Layer

The Service Layer contains reusable business logic shared across multiple controllers, keeping controllers thin and focused.

| Service           | Responsibilities                                                                                    |
|-------------------|-----------------------------------------------------------------------------------------------------|
| Gemini AI Service | Prepares prompts, communicates with Google Gemini, formats and returns AI-generated recommendations |
| JWT Service       | Creates JWT tokens, configures expiry, verifies user identity                                       |
| Password Service  | Hashes passwords, compares credentials during login                                                 |

## 7. Database Layer

**MongoDB** stores all persistent application data, with **Mongoose** serving as the Object Data Modeling (ODM) layer that simplifies interaction between Node.js and MongoDB.

### Collections

- **Users Collection**
- **Workouts Collection**

### Advantages

- Flexible, schema-driven data modeling
  - High read/write performance
  - Simple and efficient data retrieval
  - Scales easily with growing data and users
- 

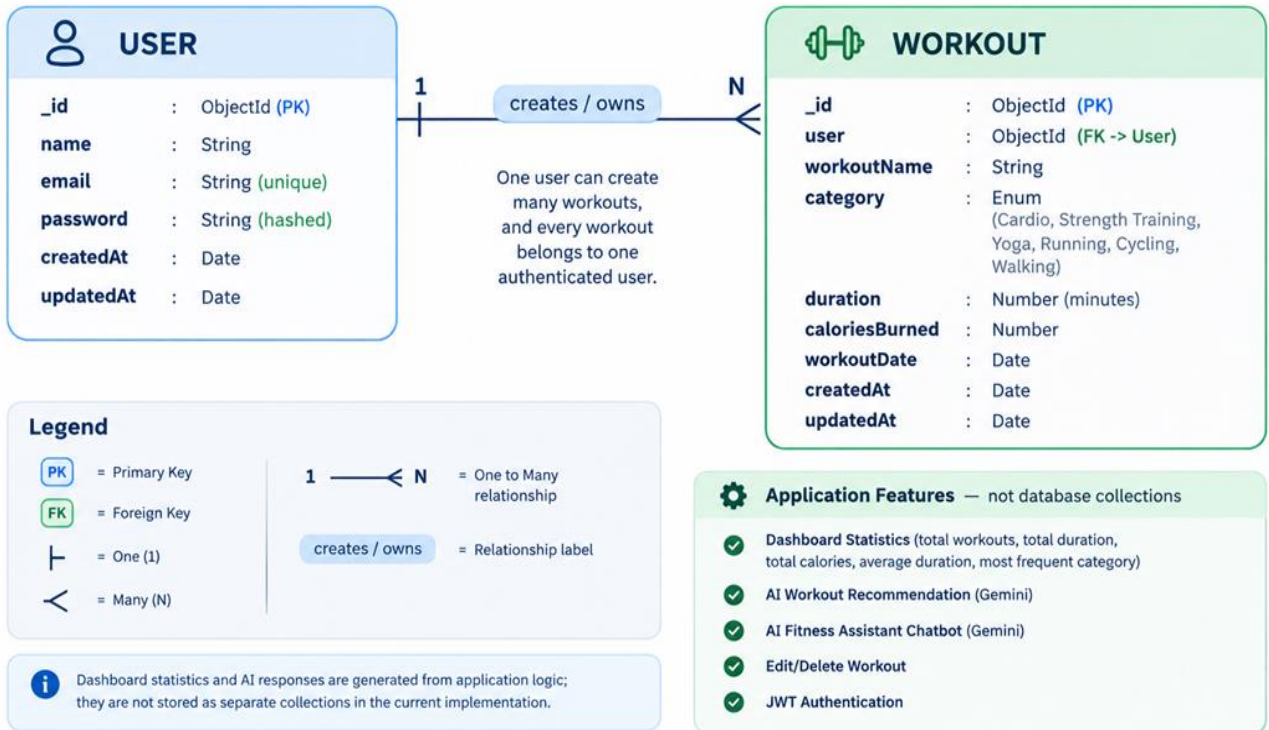
Want me to add a **request-flow diagram** (visualizing how a request travels through these seven layers), or should I now compile the full document — Background → Problem → Solution → AI Module → Features → Architecture — into one cohesive file (e.g., a Markdown or Word doc)?

## Entity Relationship (ER) Diagram Description

The AI FitTrack database consists of two primary collections connected through a one-to-many relationship. Each authenticated user can own multiple workout records.

# AI FITTRACK

## — DATABASE ER DIAGRAM



## 1. User Entity

The User entity stores authentication and profile information.

### Attributes

| Field | Type     | Description          |
|-------|----------|----------------------|
| _id   | ObjectId | Primary Key          |
| name  | String   | User Name            |
| email | String   | Unique Email Address |

|           |        |                      |
|-----------|--------|----------------------|
| password  | String | Encrypted Password   |
| createdAt | Date   | Record Creation Time |
| updatedAt | Date   | Last Updated Time    |

## Constraints

- Email must be unique.
- Password is encrypted using bcrypt.
- Name is mandatory.
- JWT token is generated after successful login.

## 2. Workout Entity

The Workout entity stores workout-related information for each authenticated user.

### Attributes

| Field          | Type     | Description                |
|----------------|----------|----------------------------|
| _id            | ObjectId | Primary Key                |
| user           | ObjectId | References User Collection |
| workoutName    | String   | Workout Name               |
| category       | String   | Workout Category           |
| duration       | Number   | Duration (Minutes)         |
| caloriesBurned | Number   | Calories Burned            |
| workoutDate    | Date     | Workout Date               |
| createdAt      | Date     | Created Timestamp          |

|           |      |                   |
|-----------|------|-------------------|
| updatedAt | Date | Updated Timestamp |
|-----------|------|-------------------|

## Relationship

### User → Workout

**Relationship Type:** One-to-Many

A single user can create multiple workout records. Each workout belongs to only one authenticated user.

Example:

John Doe

- Morning Running
- Evening Jogging
- Upper Body Strength
- Cycling Session

This relationship is maintained using the **user** field inside the Workout schema, which stores the ObjectId of the corresponding user.

# Key Features

The FitTrack AI provides a complete backend solution for workout tracking and AI-assisted fitness guidance.

## 1. Secure Authentication

The application implements JWT-based authentication.

Features include:

- User Registration
- Secure Login
- Password Encryption
- JWT Generation
- Protected APIs

## 2. Workout Management

Users can perform complete CRUD operations.

#### Supported Operations

- Add Workout
- View Workout
- Update Workout
- Delete Workout

Each workout is linked to its authenticated user.

### 3. Workout History and Management

Users can manage their workout history through the frontend and REST APIs:

- Workout Name
- Workout Category
- Workout Date

This makes workout history easier to review and maintain.

### 4. Dashboard and Progress Statistics

- Most Frequent Workout Category
- Average Workout Duration
- Total Calories Burned
- Total Duration
- Total Workouts

The dashboard provides an at-a-glance summary of the user's workout activity.

The backend communicates with Google Gemini AI to generate intelligent workout recommendations.

#### Input

- Age
- Fitness Goal
- Experience Level

## Output

- Weekly Workout Plan
- Suggested Exercises
- Training Tips
- Safety Recommendations

## 5. Google Gemini AI Recommendation

The AI module analyzes workout statistics.

## Input

- Total Workouts
- Average Duration
- Calories Burned

## Output

- Performance Analysis
- Improvement Suggestions
- Motivational Advice
- Fitness Progress Summary

## 6. AI Fitness Insights and Chatbot

- Display the conversation in a chat interface
- Generate a concise Gemini AI response
- Send the question securely to the protected backend
- Accept natural-language fitness questions

Chatbot capabilities include:

The application provides both AI-generated fitness insights from workout statistics and an interactive AI Fitness Assistant chatbot for fitness-related questions.

The application returns meaningful JSON error responses for:

- Invalid Credentials
- Missing Authentication Token
- Validation Errors

- Database Errors
- Internal Server Errors

## 7. Centralized Error Handling

The backend follows REST principles.

Supported HTTP Methods

- GET
- POST
- PUT
- DELETE

Responses are returned in JSON format.

## 8. RESTful API Design

The project separates responsibilities into

- Models
- Controllers
- Routes
- Middleware
- Services
- Configuration

This improves maintainability and scalability.

# Roles and Responsibilities

The AI FitTrack application supports one primary role: the authenticated user.

### **Authenticated User**

#### **Responsibilities**

- Register an account.
- Login securely.
- View profile information.
- Add workout records.
- View workout history.

- Search workouts.
- Update workouts.
- Delete workouts.
- Request AI workout recommendations.
- Request AI fitness insights.

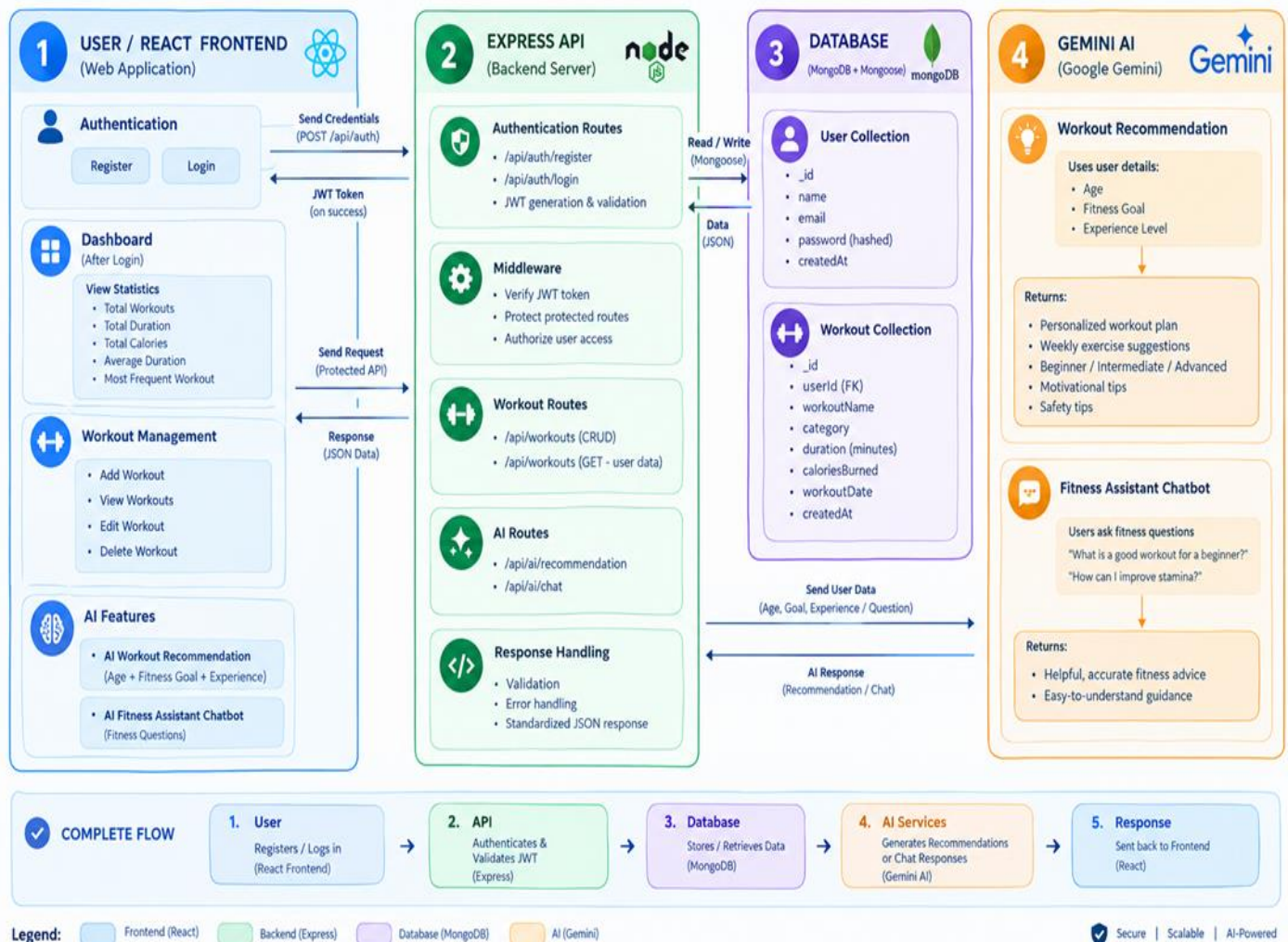
## Permissions

| Operation         | Permission |
|-------------------|------------|
| Register          | Yes        |
| Login             | Yes        |
| View Profile      | Yes        |
| Add Workout       | Yes        |
| Get Workouts      | Yes        |
| Search Workouts   | Yes        |
| Update Workout    | Yes        |
| Delete Workout    | Yes        |
| AI Recommendation | Yes        |
| Fitness Insights  | Yes        |



# AI FITTRACK – APPLICATION FLOW DIAGRAM

Fitness Tracking | Smart Recommendations | AI-Powered Assistance

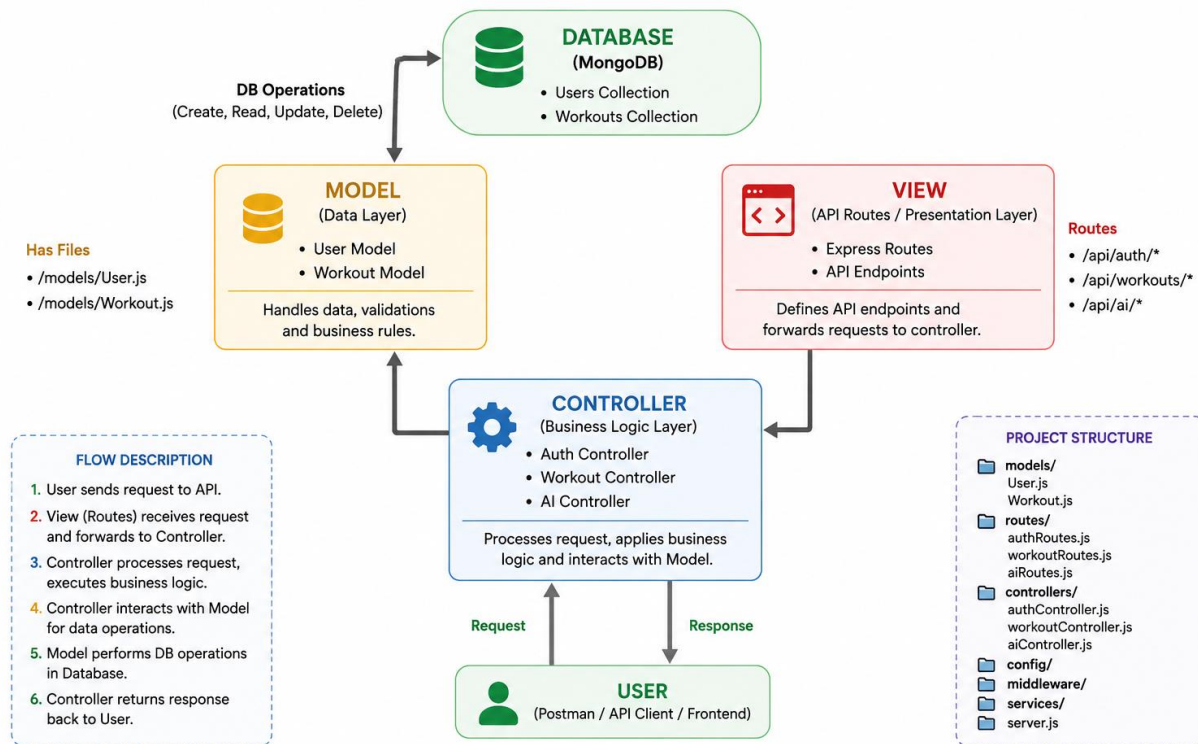


## User Flow

## MVC Architecture Pattern

The FitTrack AI is designed using the **Model–View–Controller (MVC)** architecture pattern. This architecture separates application logic into independent components, improving code organization, maintainability, and scalability.

## AI FITTRACK API (BACKEND) – MVC PATTERN



### Model Layer (Data Layer)

The Model Layer defines the structure of the application's data using Mongoose schemas. It communicates directly with MongoDB and performs database operations.

Models in the project include:

- User Model
- Workout Model

Responsibilities:

- Schema Definition
- Data Validation
- CRUD Operations
- Collection Mapping

### View Layer (API Interface)

In the completed project, the View Layer is represented by the React frontend. It provides the login and registration interface, dashboard, workout forms, workout history cards, AI recommendation form, and AI Fitness Assistant chat interface. The frontend communicates with the Express backend through protected REST APIs.

Responsibilities:

- Send Requests
- Receive JSON Responses
- Display Data
- Handle Client-side Interaction

## **Controller Layer (Business Logic)**

The Controller Layer acts as the intermediary between the View and Model. It processes incoming requests, validates input, invokes services and models, and sends structured JSON responses back to the client.

Responsibilities:

- Request Validation
- Business Logic Execution
- Database Interaction
- AI Service Invocation
- Response Generation

The separation of these three layers ensures loose coupling, simplifies testing, enhances code readability, and makes the application easier to extend with future features.

# **Epic 2: Project Setup and Configuration**

The FitTrack AI project follows a well-structured backend architecture that separates business logic, routing, database operations, middleware, and AI services into individual modules. This modular design improves code readability, maintainability, and scalability while making the project easier to debug and extend.

The application is developed using React for the frontend, Node.js and Express.js for the backend, MongoDB for persistent data storage, and Google Gemini AI for recommendations, insights, and chatbot responses.

## **Creating Project Folder**

## Step 1: Create the Project Folder

Create a new folder with the project name.

AI-FitTrack-v2

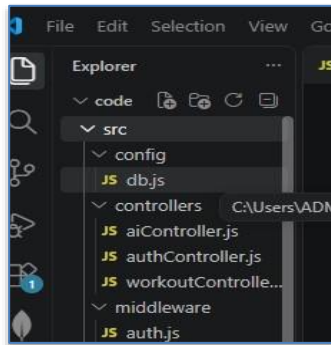
## Step 2: Open the Project

Open the project folder in **Visual Studio Code**.

## Step 3: Create the Server Folder

Inside the project directory, create a folder named:

Backend



Backend source code is maintained inside this folder, while the React frontend is maintained separately.

## Step 4: Initialize Node Project

Open the terminal inside the **server** folder and execute:

```
npm init -y
```

This command generates the **package.json** file containing project metadata and dependency information.

## Step 5: Install Required Packages

Install the required dependencies.

```
npm install express mongoose dotenv bcryptjs jsonwebtoken cors morgan @google/genai
```

Install Nodemon as a development dependency.

```
npm install --save-dev nodemon
```

## Environment Variables

Sensitive information is stored in the **.env** file.

Example

```
PORT=5000
```

```
MONGO_URI=mongodb://localhost:27017/fittrack
```

```
JWT_SECRET=my_secret_key
```

```
GEMINI_API_KEY=your_google_gemini_api_key
```

```
GEMINI_MODEL=your_available_gemini_model
```

Using environment variables improves security by preventing sensitive information from being exposed in source code.

## Epic 3: Backend Development

Backend development focuses on implementing secure REST APIs for authentication, workout CRUD operations, dashboard statistics, AI recommendations, fitness insights, and the AI Fitness Assistant chatbot.

The project is divided into multiple modules that collectively perform authentication, workout management, AI integration, and database operations.

Major modules include:

- Server Configuration
- Database Configuration
- Authentication
- Workout Management
- AI Recommendation
- Middleware
- Error Handling

Video Link:

<https://drive.google.com/file/d/1KswrctSaPI0eOYyJ3GkbbVHvHsU3EfZJ/view?usp=sharing>

## Epic 4: Database Configuration

The FitTrack AI uses **MongoDB** as its primary NoSQL database for storing user credentials and workout information. **Mongoose** serves as the Object Data Modeling (ODM) library, enabling seamless interaction between the application and the MongoDB database. Mongoose provides schema validation, middleware support, data relationships, and simplified CRUD operations.

The database layer is responsible for maintaining data integrity, validating incoming requests, and ensuring secure storage of user information.

## MongoDB Configuration

MongoDB stores application data in collections rather than traditional relational database tables. The FitTrack AI maintains separate collections for users and workouts.

### Database Collections

- Users Collection
- Workouts Collection

Each collection stores JSON-like documents (BSON format), allowing flexible schema design while maintaining efficient data retrieval.

### Installing Mongoose

Install Mongoose using npm.

```
npm install mongoose
```

Mongoose acts as the bridge between the Node.js application and MongoDB by providing schema definitions, validation, and database methods.

### MongoDB Connection

The database connection is configured inside the **config/db.js** file. This module establishes a connection to the MongoDB server when the application starts.

```
const mongoose = require('mongoose');

const connectDB = async () => {
  try {
    const conn = await mongoose.connect(process.env.MONGO_URI);
    console.log(`MongoDB Connected: ${conn.connection.host}`);
  } catch (error) {
    console.error(`Error connecting to MongoDB: ${error.message}`);
    process.exit(1);
  }
};

module.exports = connectDB;
```

## Responsibilities

- Connect to MongoDB
- Handle Connection Errors
- Display Connection Status
- Export Database Connection

## Advantages

- Centralized database configuration
- Reusable connection module
- Automatic connection management
- Simplified error handling

# Mongoose Schemas

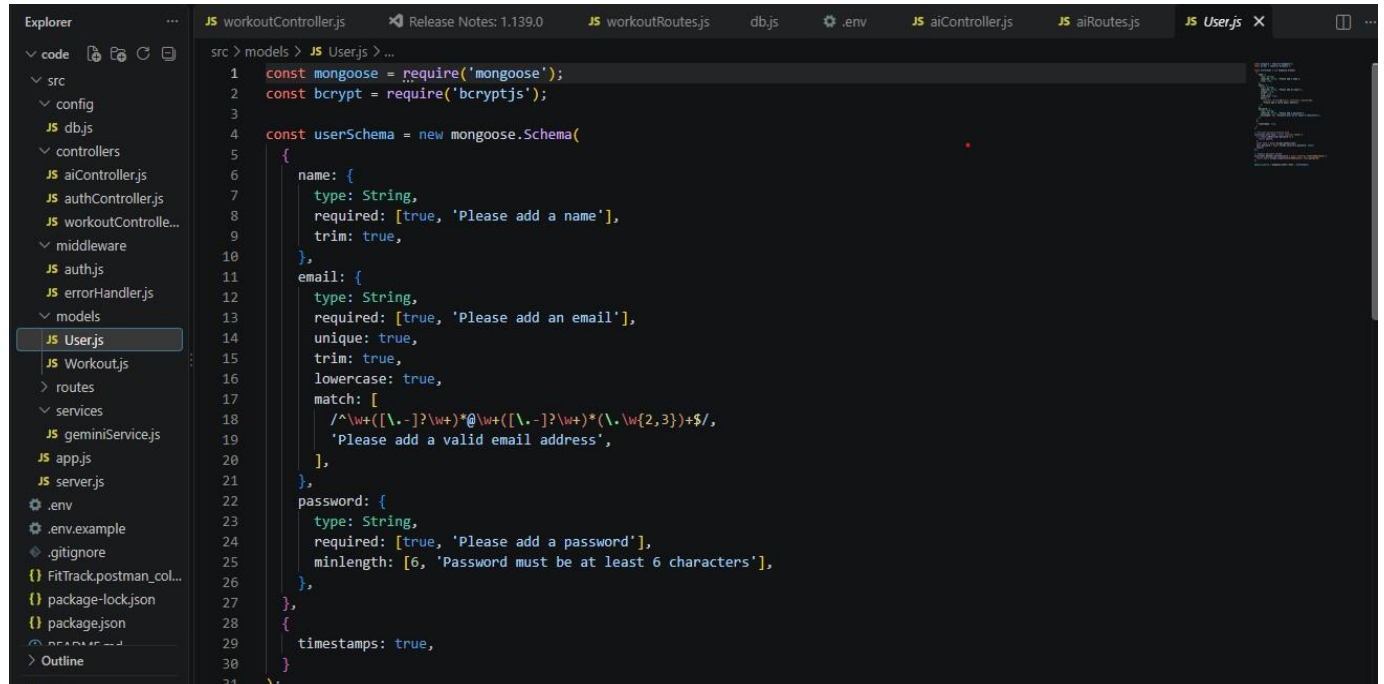
The AI FitTrack backend contains two primary Mongoose schemas:

1. User Schema
2. Workout Schema

Schemas define the structure, validation rules, and relationships for MongoDB documents.

## 1. User Schema (models/User.js)

The User schema stores authentication information for registered users.

A screenshot of a code editor with a dark theme. The Explorer sidebar on the left shows a file tree with folders like 'code', 'src', 'config', and 'models'. The 'models' folder is expanded, and 'User.js' is selected. The main editor area shows the content of 'User.js'. The code defines a Mongoose schema for a user, including fields for name, email, and password, each with specific validation rules like required, unique, trim, and minlength. It also includes a regex for email validation and enables timestamps.

```
1 const mongoose = require('mongoose');
2 const bcrypt = require('bcryptjs');
3
4 const userSchema = new mongoose.Schema(
5   {
6     name: {
7       type: String,
8       required: [true, 'Please add a name'],
9       trim: true,
10    },
11    email: {
12      type: String,
13      required: [true, 'Please add an email'],
14      unique: true,
15      trim: true,
16      lowercase: true,
17      match: [
18        /^\.w+([\.-]?w+)*@w+([\.-]?w+)*(\.w{2,3})+$/ ,
19        'Please add a valid email address',
20      ],
21    },
22    password: {
23      type: String,
24      required: [true, 'Please add a password'],
25      minlength: [6, 'Password must be at least 6 characters'],
26    },
27  },
28  {
29    timestamps: true,
30  }
31 );
```

### Validation Rules

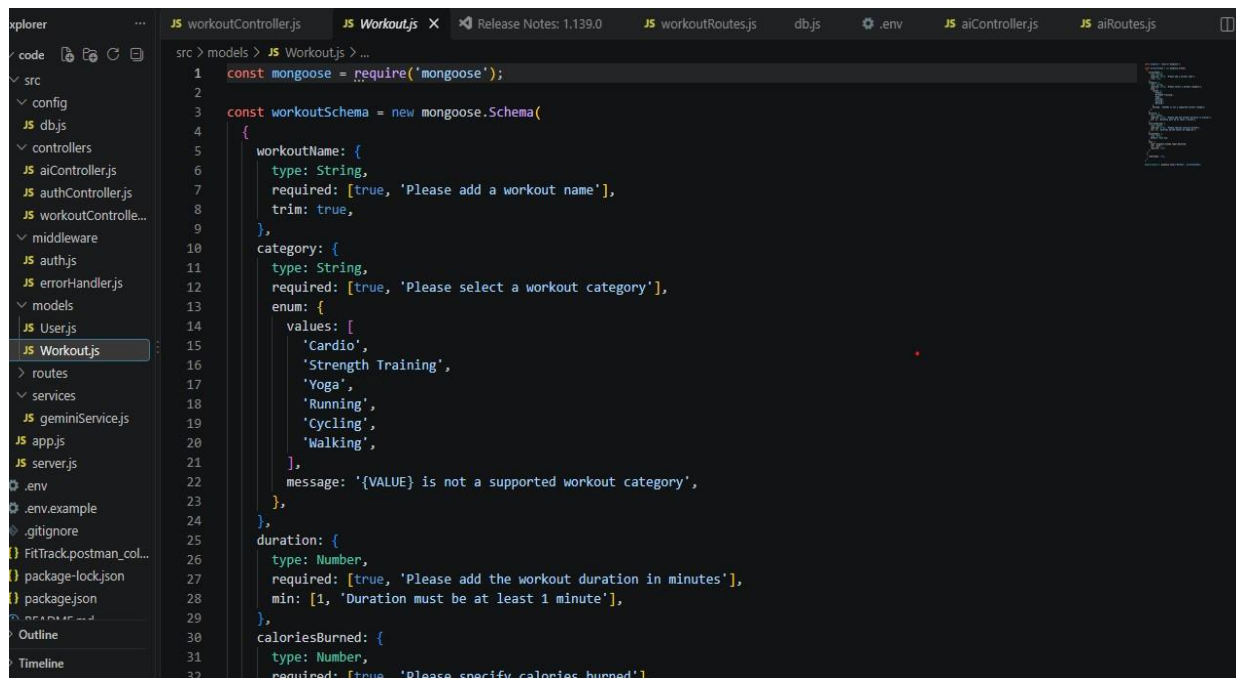
- Name cannot be empty.
- Email must be unique.
- Password is mandatory.
- Password is encrypted using bcrypt.js before saving.
- Automatic timestamps are enabled.

### User Schema Features

- Email uniqueness validation
- Password hashing
- Automatic timestamps
- Secure authentication support
- JWT integration

## 2. Workout Schema (models/Workout.js)

The Workout schema stores workout information created by authenticated users.



```
1  const mongoose = require('mongoose');
2
3  const workoutSchema = new mongoose.Schema({
4
5    workoutName: {
6      type: String,
7      required: [true, 'Please add a workout name'],
8      trim: true,
9    },
10   category: {
11     type: String,
12     required: [true, 'Please select a workout category'],
13     enum: {
14       values: [
15         'Cardio',
16         'Strength Training',
17         'Yoga',
18         'Running',
19         'Cycling',
20         'Walking',
21       ],
22       message: '{VALUE} is not a supported workout category',
23     },
24   },
25   duration: {
26     type: Number,
27     required: [true, 'Please add the workout duration in minutes'],
28     min: [1, 'Duration must be at least 1 minute'],
29   },
30   caloriesBurned: {
31     type: Number,
32     required: [true, 'Please specify calories burned'],
```

## Validation Rules

- Workout Name is mandatory.
- Category is mandatory.
- Duration must be greater than zero.
- Calories Burned must be greater than or equal to zero.
- Workout Date is required.
- Every workout belongs to one authenticated user.

## Workout Schema Features

- Schema validation
- Automatic timestamps
- User relationship
- Easy search support
- Efficient CRUD operation

# Epic 5: Project Execution

- Install Postman or ThunderClient (VS Code extension).
- Start the backend server:

“npm run dev”

## API Testing

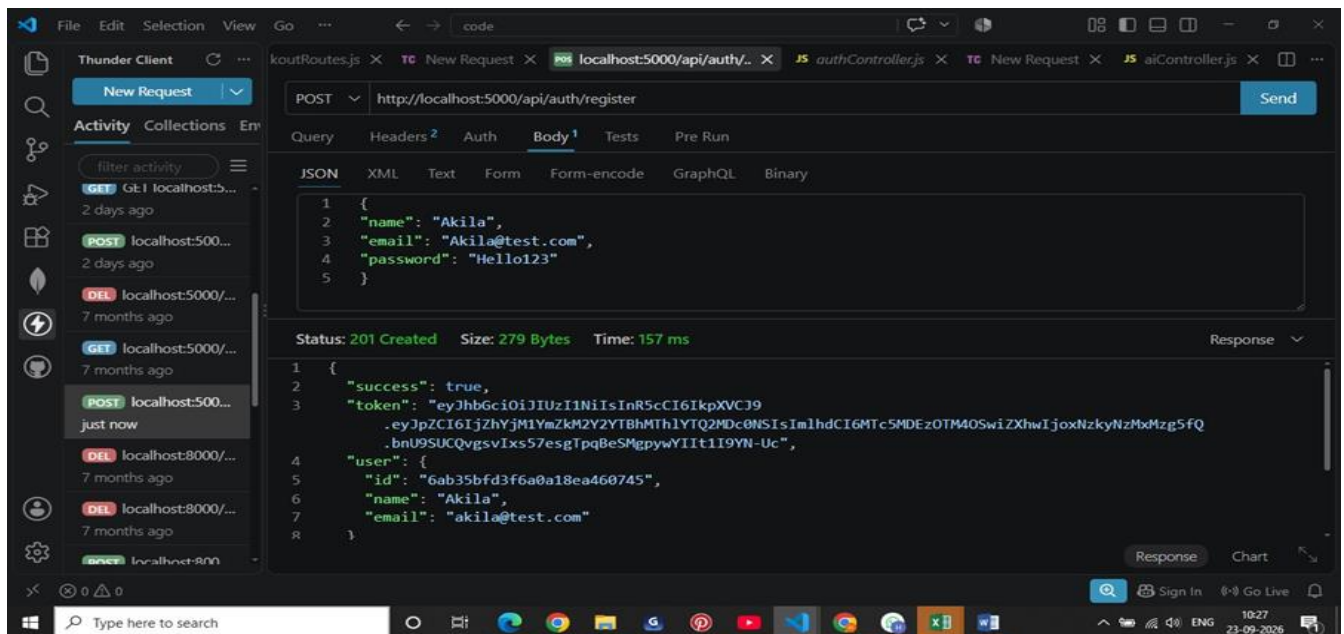
API testing ensures that every endpoint functions correctly and returns the expected response. The FitTrack AI is tested using **Postman**, allowing developers to verify request handling, authentication, database operations, and AI integration.

Testing is performed in the following order:

1. Authentication APIs
2. Workout APIs
3. AI APIs

# Authentication API Testing

## Register User

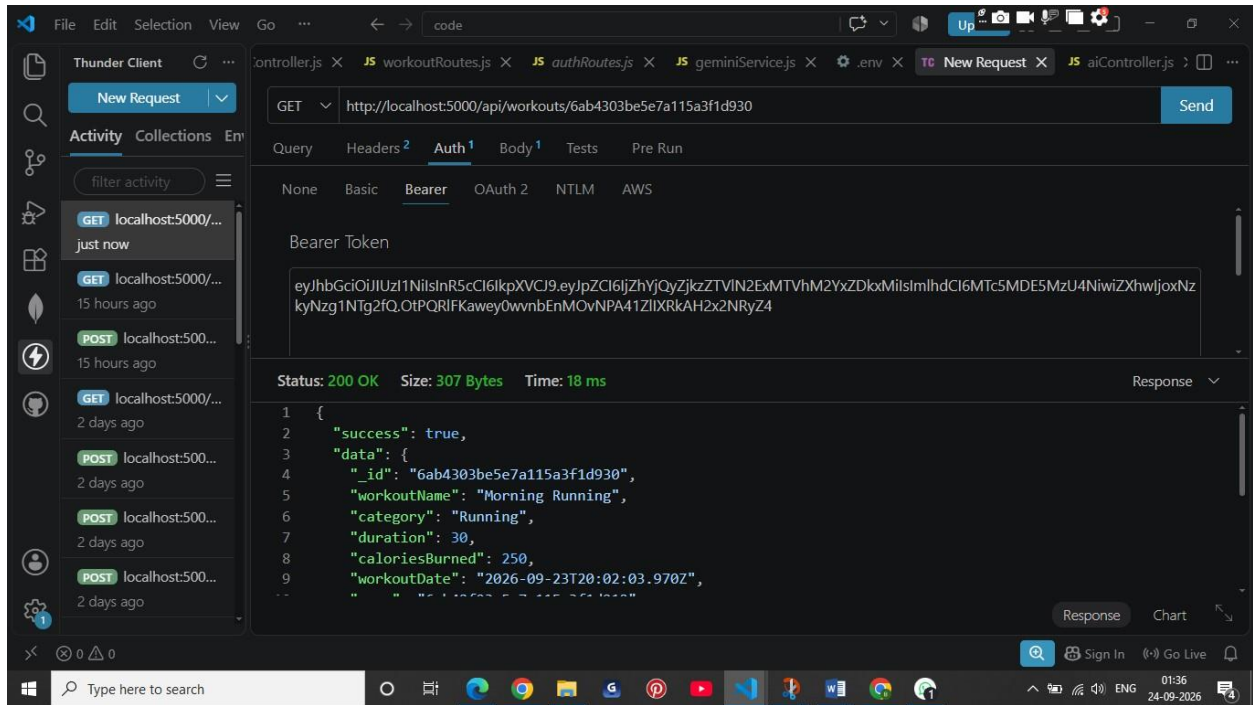


## Login User

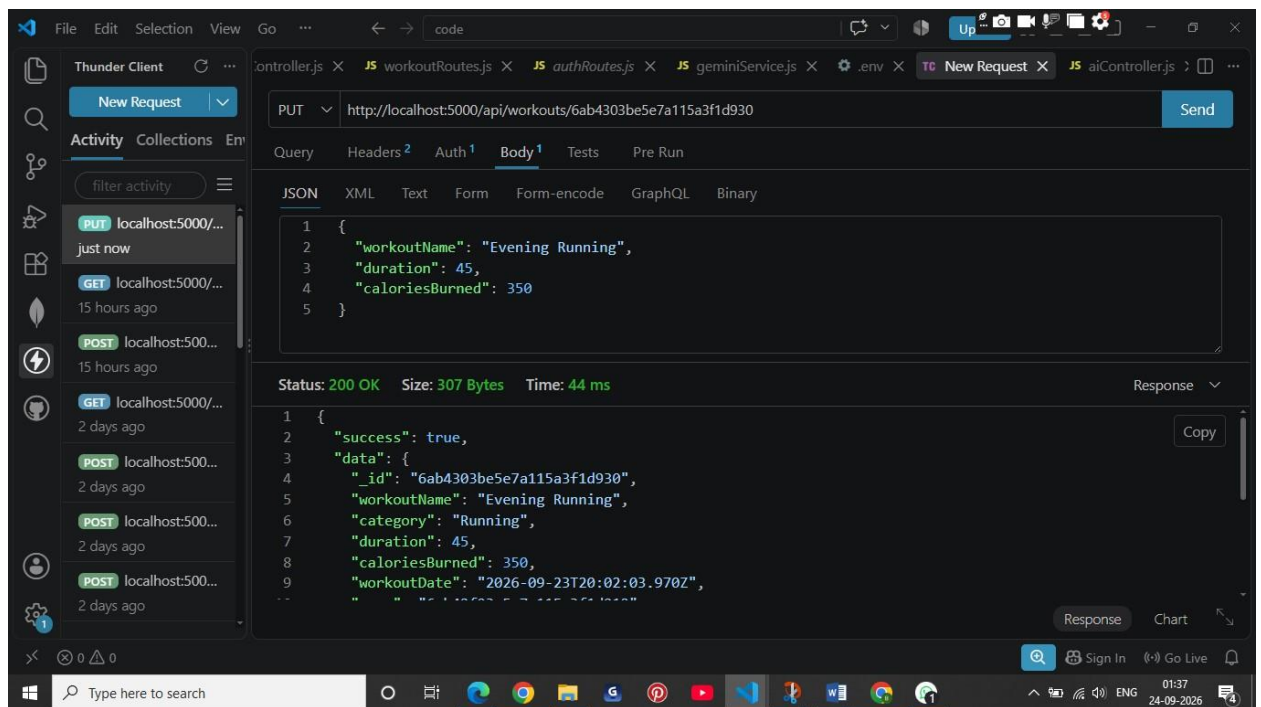




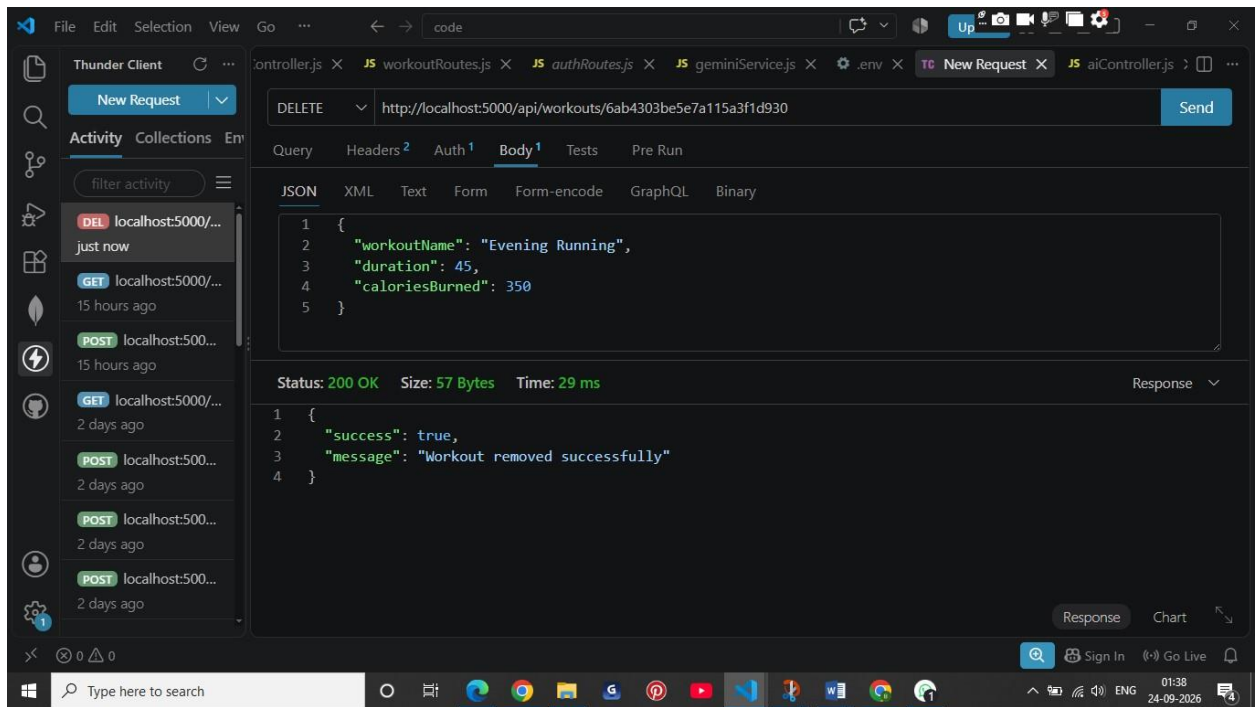
## Get Workout By ID



## Update Workout

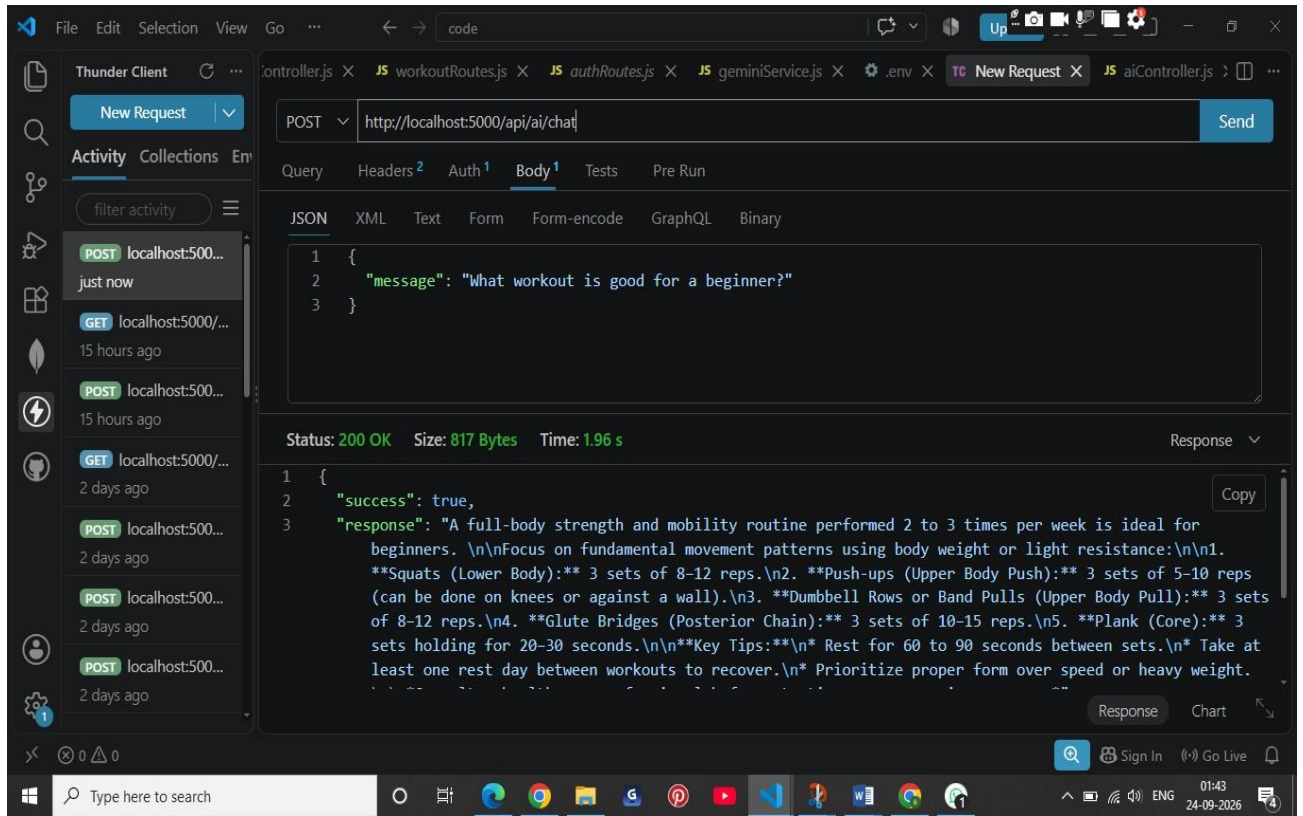


## Delete Workout

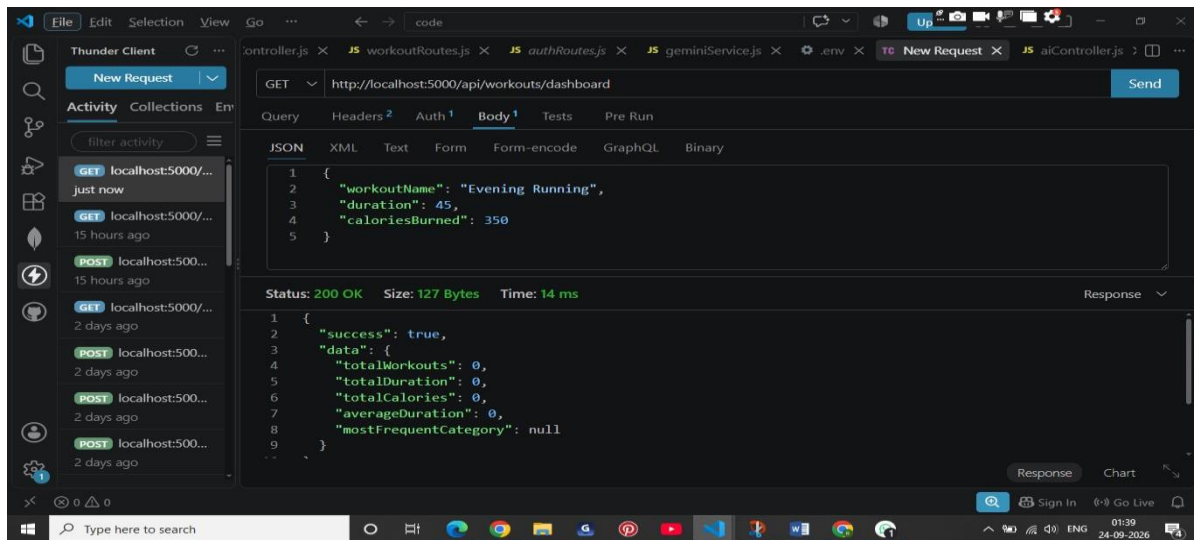


## AI API and Chatbot Testing

## Workout Recommendation

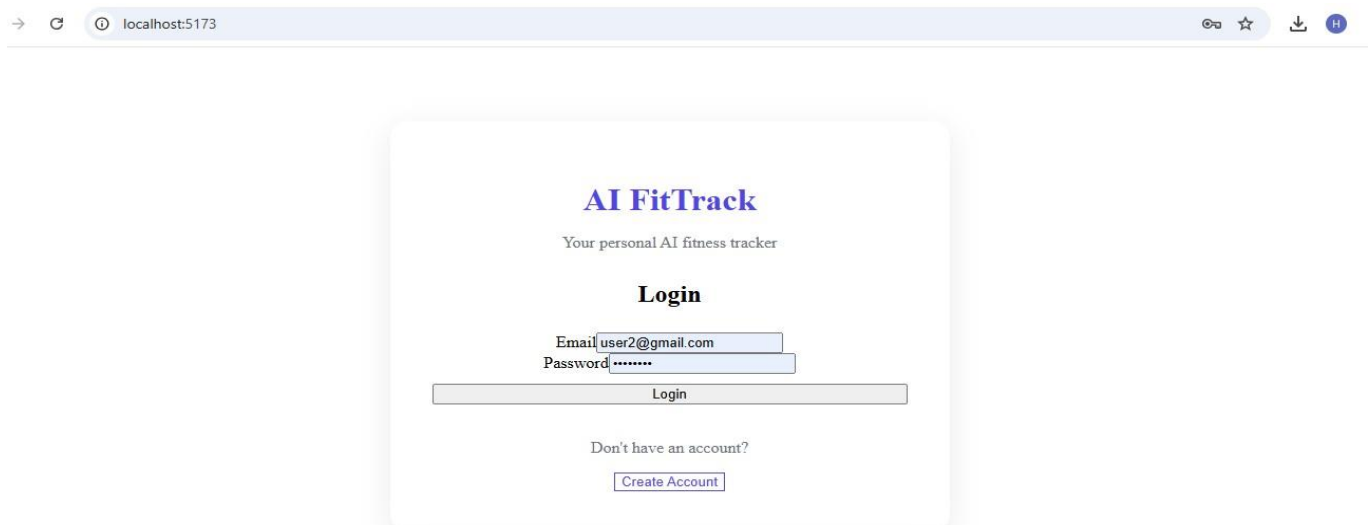


**DashBoard(new feature)**

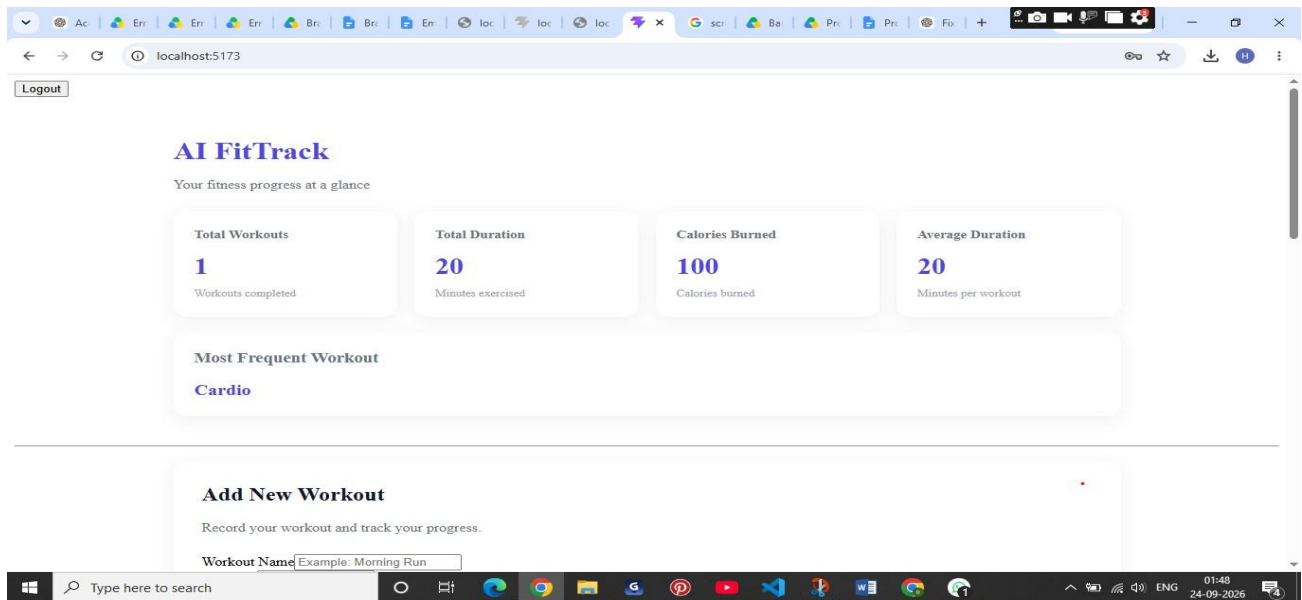


## Full FitTrack API-webApplication

### 1.Login



## 2. Dashboard



## 2.Ai workout recommendation

The screenshot shows the AI Workout Recommendation form and its output. The form has fields for 'Age' (18), 'Fitness Goal' (abs), and 'Experience Level' (Beginner). A 'Generate AI Recommendation' button is located below the form. The result is displayed in a light blue box with the title 'Your Personalized Recommendation'. The recommendation text states: 'To build visible abs as an 18-year-old beginner, you must combine core-strengthening exercises with a caloric deficit, because abs only show when body fat is low. Perform the following routine three times a week with at least one rest day between sessions. Complete each exercise for 30 to 45 seconds, rest for 15 seconds, and repeat the entire circuit for three total rounds. Start the routine with standard crunches to target the upper abs, then move into leg raises to engage the lower abs, followed by forearm planks to build core stability, and finish with bicycle crunches to work the obliques. In addition to this core work, engage in brisk walking, running, or any sport for 30 minutes daily to burn body fat, and drink primarily water while cutting out sugary drinks.'

AI Workout Recommendation

Get a personalized workout plan using AI.

Age: 18  
Fitness Goal: abs  
Experience Level: Beginner

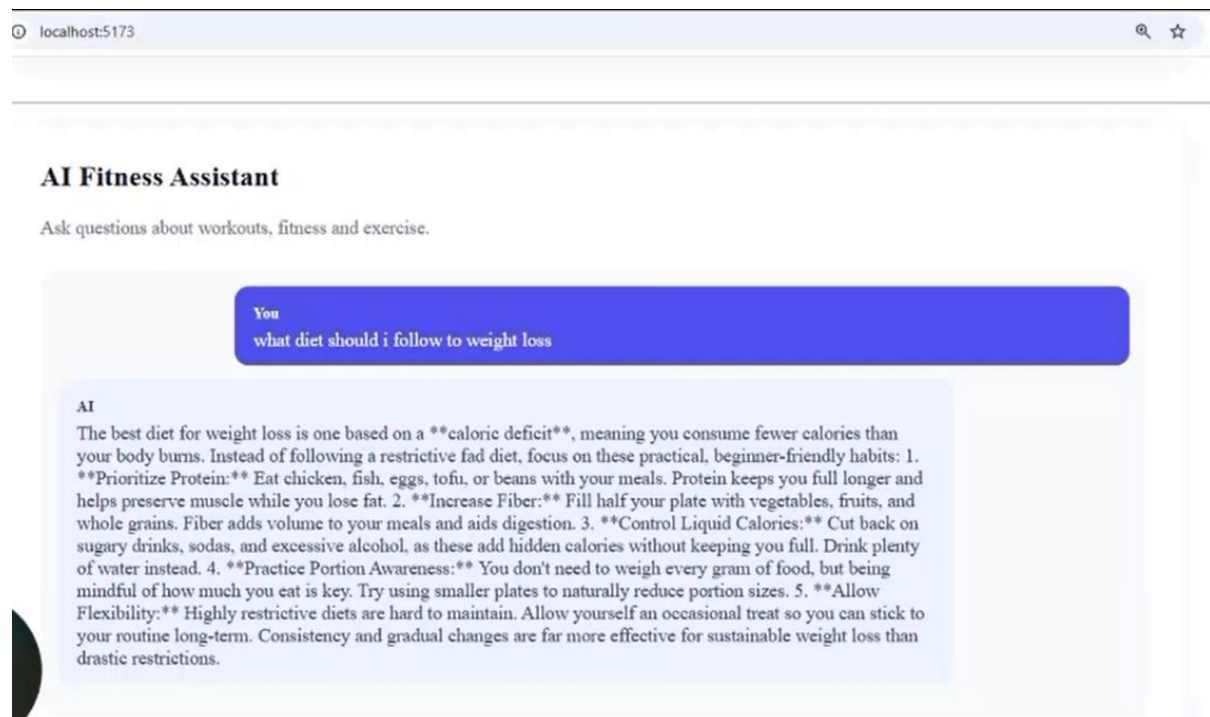
Generate AI Recommendation

**Your Personalized Recommendation**

To build visible abs as an 18-year-old beginner, you must combine core-strengthening exercises with a caloric deficit, because abs only show when body fat is low. Perform the following routine three times a week with at least one rest day between sessions. Complete each exercise for 30 to 45 seconds, rest for 15 seconds, and repeat the entire circuit for three total rounds.

Start the routine with standard crunches to target the upper abs, then move into leg raises to engage the lower abs, followed by forearm planks to build core stability, and finish with bicycle crunches to work the obliques. In addition to this core work, engage in brisk walking, running, or any sport for 30 minutes daily to burn body fat, and drink primarily water while cutting out sugary drinks.

### 3. Personal Ai-Fitness Assistant



#### Demo Video Link:

[https://drive.google.com/file/d/12I45pBcODIHm3yhM6XqUF0NZlxSWDmtM/view?usp=drive\\_link](https://drive.google.com/file/d/12I45pBcODIHm3yhM6XqUF0NZlxSWDmtM/view?usp=drive_link)

API TESTING VIDEO : [https://drive.google.com/file/d/1TCXG48a-78ryxF3gipdRMwRBwuhAgPaa/view?usp=drive\\_link](https://drive.google.com/file/d/1TCXG48a-78ryxF3gipdRMwRBwuhAgPaa/view?usp=drive_link)

#### CodeDrive:

[https://drive.google.com/drive/folders/11DghjYP8Yp09wUsZjKRitLmhSOESBslj?usp=drive\\_link](https://drive.google.com/drive/folders/11DghjYP8Yp09wUsZjKRitLmhSOESBslj?usp=drive_link)

